The analysis of algorithm efficiency

February 24, 2026

1 Introduction

In general, when analyzing an algorithm, we are interested in two key aspects:

1. **Correctness** (Does it work?)
2. **Performance** (Does it have a good performance?)

We will explore in this lecture how to formally answer these questions, on a case study of the sorting problem. We will start with InsertionSort and move to the more complicated MergeSort. Generally, a sorting problem asks that given an unordered input, it be sorted according to some criteria.

SORTING
INPUT: A sequence (or array) of n elements drawn from a totally ordered set (meaning there exists a defined comparison operation, like $\leq$), $a = \langle a_0, a_1, ..., a_{n-1} \rangle$;
OUTPUT: $a' = \langle a'_0, a'_1, \ldots, a'_{n-1} \rangle$, a permutation of the elements in a, such that $a'_i \; R \; a'_{i+1}$, $\forall \; 0 \leq i < n-1$, where R is an ordered relationship.

We will consider in the following sections the problem of sorting nonnegative integers in increasing order, where the input sequence comes in the form of an array with n elements.

Example
Input: $a = [5, 2, 4, 3, 6, 1, 3]$
Output: $a = [1, 2, 3, 3, 4, 5, 6]$

2 Insertion sort

Insertion sort operates similarly to how people typically arrange a hand of playing cards. We start with an empty left hand and a set of face-down cards on the table. We pick up one card at a time from the table and place it into its *appropriate position* in the left hand. To determine the correct position, the card is compared with those already in the left hand. At all times, the cards held in the left hand (from the top of the face-down pile) are sorted.

Algorithm 1 gives pseudocode for the Insertion sort algorithm.

The index i indicates the "current card" being inserted into the hand. At the beginning of each iteration of the **for** loop, the subarray consisting of elements $a[0..i-1]$ constitutes the currently sorted hand, while the remaining subarray $a[i+1..n-1]$ corresponds to the pile of cards still on the table.

Algorithm 1 InsertionSort(a)

Input: An array a of nonnegative integers of length n.
Output: The sorted array (the algorithm sorts the input numbers **in place**: it rearranges the numbers within the array a, with at most a constant number of them stored outside the array at any time.)

```
n ← a.size()
for i ← 1..n − 1 do
    current ← a[i]
    //Insert a[i] into the sorted sequence a[0..i − 1] in its proper place
    j ← i − 1
    while j ≥ 0 and a[j] > current do
        //Shift all values > current one by one position to the right, to make space
        a[j + 1] ← a[j]
        j ← j − 1
    a[j + 1] ← current
```

2.1 Correctness of InsertionSort. Loop invariants.

You may think that it is obvious that the algorithm works, but we still need to prove it. We will formalize the properties of the sorted subarray $a[0..i-1]$ (which has the exact elements on the first i positions of the initial array, but in sorted order) as a **loop invariant**.

We use **loop invariants** to help us understand why an algorithm is correct. We must show three things about a loop invariant:

- **Initialization:** The invariant is true before the first iteration of the loop.
- **Maintenance:** If it is true before an iteration of the loop, it remains true after the current iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

If you'd like a mnemonic to help remember these three loop invariant properties, you can try **It Must Terminate**, or **It Must be True**, or come up with your own.

When the first two properties hold, the loop invariant is true prior to every iteration of the loop. There is a similarity to **mathematical induction**, where to prove that a property holds, one proves a base case and an inductive step. Here, the initialization corresponds to the base case, and the maintenance property corresponds to the inductive step. The termination property differs from how we usually use mathematical induction (apply the inductive step infinitely) in that we stop the "induction" when the loop terminates.

Let us see how these properties hold for insertion sort.

Initialization (Base Case): $i = 1$, and the subarray $a[0..i-1]$ contains only one element, $a[0]$, and is thus sorted. So the loop invariant holds prior to the first iteration of the loop.

Maintenance (Inductive Step): Next, we show that each iteration maintains the loop invariant: after iteration i, the subarray $a[0..i]$ is sorted.

Upon entering the i-th iteration, we know that $a[0..i-1]$ is sorted. Informally, on iteration i, the body of the for loop works by shifting the elements $a[i-1], a[i-2], \ldots$ one position to the right until it finds the proper position for $a[i]$ and inserts it there. Let j' be this index. We have that $a[j'] \leq a[i] < a[j'+1]$. The elements $a[j'+1..i-1]$ were all shifted by one position

to $a[j'+2..i]$, we assign $a[i]$ to $a[j'+1]$, and we know that $a[0..j']$ is sorted by the maintenance property. We thus obtain a sorted subarray $a[0..i]$

Termination (Conclusion): The condition causing the for loop to terminate is $i = n$ (Note that $i \leftarrow 1..n-1$ is a different notation for $i = 1; i < n; i++$).

If we substitute i by n in the loop invariant, we get $a[0..n-1]$ to be sorted.

Conclusion: Since this is the entire array, the array is completely sorted, thus the algorithm is correct.

2.2 Running time of InsertionSort

The running time of InsertionSort is about n^2 operations: at each iteration i, the algorithm may have to move all elements $0..i-1$, which totals $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ operations.

Now that we have a correct algorithm, the next question is: **Can we do (asymptotically) better** than n^2? We will see next that Merge sort will scale to $n\log(n)$, which is much faster. But first, more on algorithm analysis.

3 Analyzing algorithms

Analyzing an algorithm means predicting the resources that the algorithm requires. Most often it is **computational time** that is the main concern, but analyses of memory or communication bandwidth can also be made. By analyzing several candidate algorithms for a problem, we can generally identify a most efficient one. There may be more than one viable candidate, but some algorithms can often be discarded as inferior.

3.1 Model of computation

Before analyzing an algorithm, we must have a model of the implementation technology that we will use, including a model for the resources of that technology and their costs. Like in [1], we shall assume a generic one-processor, random-access machine (RAM) model of computation, and understand that our algorithms will be implemented as computer programs. In the RAM model, instructions are executed one after another, with no concurrent operations.

We will assume that the RAM model contains instructions commonly found in real computers: arithmetic (addition, subtraction, multiplication, division, remainder, etc.), data movement (load, store, copy), and control (conditional and unconditional branch, subroutine call and return). Each such instruction takes a constant amount of time.

The data types in the RAM model are integer and floating point (for storing real numbers). We also assume a limit on the size of each word of data. For example, when working with inputs of size n, we typically assume that integers are represented by $c \lg n$ bits for some constant $c \geq 1$. The requirement $c \geq 1$ ensures that each word can hold the value of n, so we can index the individual input elements, but at the same time the word size does not grow arbitrarily.

3.2 Running time and input size

Generally, the time taken by an algorithm grows with the size of its input, so it is traditional to describe the running time of a program as a function of the size of its input.

The **input size** depends on the problem being studied. For many problems (sorting, searching), the most natural measure is the *number of items* in the input (e.g., the array size for sorting). For many other problems, like multiplying two integers, the best measure of input size is the *total number of bits* needed to represent the input in ordinary binary notation. For many

problems yet, it is more appropriate to describe the size of the input with two numbers instead of one. For example, if we are analyzing a graph problem, the input size can be described by the number of vertices and the number of edges in the graph.

When analyzing an algorithm for a problem, it is important to identify which input size measure is most suited.

The **running time** of an algorithm on a particular input is *the number of "steps"* (primitive operations) executed. For the moment, we will assume that a constant amount of time is required to execute each line of our pseudocode. Different lines may take different amounts of time. We assume that line i takes constant time c_i. This viewpoint is consistent with the RAM model, and it also reflects how the pseudocode would be implemented on most computers.

3.3 Analysis of Insertion sort

Algorithm 2 Insertion-Sort (a)

Line	Algorithm	cost	no. of times
1:	$n \leftarrow a.size()$	c_1	1
2:	**for** $i \leftarrow 1..n-1$ **do**	c_2	n
3:	$\quad current \leftarrow a[i]$	c_3	$n-1$
4:	$\quad j \leftarrow i-1$	c_4	$n-1$
5:	$\quad$ **while** $j \geq 0$ **and** $a[j] > current$ **do**	c_5	$\sum_{i=1}^{n-1} t_i$
6:	$\quad\quad a[j+1] \leftarrow a[j]$	c_6	$\sum_{i=1}^{n-1}(t_i - 1)$
7:	$\quad\quad j \leftarrow j-1$	c_7	$\sum_{i=1}^{n-1}(t_i - 1)$
8:	$\quad a[j+1] \leftarrow current$	c_8	$n-1$

For each line of the algorithm 2 we denote by c_i the cost of the line (in terms of number of steps), and specify the number of times each line is executed. t_i denotes the number of times the while loop test in line 5 is executed for that value of i. Note that the test for the for and while loops is executed one time more than the loop body (namely the check that ends the loop).

The running time of the algorithm is the sum of running times for each statement executed. If a statement takes c_i steps to execute and is executed n times, it will contribute $c_i \cdot n$ to the total running time. The total running time of InsertionSort will be:

$$T(n) = c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5 \sum_{i=1}^{n-1} t_i + c_6 \sum_{i=1}^{n-1}(t_i - 1) + c_7 \sum_{i=1}^{n-1}(t_i - 1) + c_8(n-1).$$

The running time of the algorithm does not only depend on the size of the input given. For inputs of the same size, it matters *which* input is given. For InsertionSort, the **best case** is when the input array is already sorted. This means no shifts are necessary within the array to construct the solution. The **while** loop is only executed once for each value of i, so the best case running time becomes:

$$\begin{aligned} T(n) &= c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5 \sum_{i=1}^{n-1} 1 + c_6 \sum_{i=1}^{n-1} 0 + c_7 \sum_{i=1}^{n-1} 0 + c_8(n-1) \\ &= c_1 + c_2 n + c_3 n - c_3 + c_4 n - c_4 + c_5(n-1) + c_8 n - c_8 \\ &= n(c_2 + c_3 + c_4 + c_5 + c_8) + c_1 - (c_3 + c_4 + c_5 + c_8) \end{aligned}$$

In this case we can express $T(n)$ as $an + b$, with constants a and b depending on c_i, thus the running time is a **linear function** of input size n.

The **worst case** for InsertionSort is when the array is sorted in decreasing order. In this case, the **while** loop will execute i times for each $i = 1..n-1$, thus $t_i = i$. Note that $\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$. We get that the running time of InsertionSort in the worst case is:

$$\begin{aligned}
T(n) &= c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5 \sum_{i=1}^{n-1} i + c_6 \sum_{i=1}^{n-1}(i-1) + c_7 \sum_{i=1}^{n-1}(i-1) + c_8(n-1) \\
&= c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5 \frac{n(n-1)}{2} + c_6 \frac{(n-2)(n-1)}{2} + c_7 \frac{(n-2)(n-1)}{2} + c_8(n-1) \\
&= c_1 + c_2 n + c_3 n - c_3 + c_4 n - c_4 + c_5 \frac{n^2-n}{2} + c_6 \frac{n^2-3n+2}{2} + c_7 \frac{n^2-3n+2}{2} + c_8 n - c_8 \\
&= \frac{n^2}{2}(c_5 + c_6 + c_7) + n(c_2 + c_3 + c_4 - \frac{c_5}{2} - \frac{3c_6}{2} - \frac{3c_7}{2} + c_8) + c_1 + c_6 + c_7 - (c_3 + c_4 + c_8)
\end{aligned}$$

We can express T(n) in the worst case as $an^2 + bn + c$, thus it is a **quadratic function** of the input size n.

3.4 Worst-case and average-case analysis

We looked so far at the best case and worst case for InsertionSort. We usually concentrate on finding the worst-case running time, that is, the longest running time for any input of size n. The worst-case running time gives an upper bound on the running time for any input. Knowing this upper bound provides a guarantee that the algorithm will never take any longer.

For some algorithms, the worst case may occur quite often. For example, searching for information that is not in the input data may be frequent in some applications.

The **average case** is often roughly as bad as the worst case. Suppose that we randomly choose n numbers and apply insertion sort. How long will it take to find where in subarray $a[0..i-1]$ to insert element $a[i]$? On average, half of the elements in $a[0..i-1]$ are less than $a[i]$, and half are greater. Thus, on average, we check half of the subarray $a[0..i-1]$, and so t_i is about $i/2$. The resulting average-case running time is also a quadratic function of the input size, just like the worst-case running time.

In some particular cases, we will be interested in the average-case running time of an algorithm, and apply probabilistic analysis. But more on that in the following lectures.

4 Merge sort

We discussed in the previous sections the insertion sort algorithm, which uses an **incremental** approach (we increase step by step the length of the sorted subarray). We will now consider a **Divide-and-conquer** approach, namely Merge sort.

First, let's remember the Divide-and-conquer paradigm:

- **Divide** the problem into smaller sub-problems
- **Conquer** the subproblems by (recursively) solving them. Small enough subproblems are solved straightforward, thus stopping the recursion.
- **Combine** the solutions of the subproblems to get the solution of the initial problem.

The merge sort algorithm operates in the following manner:

- Divide the n-length array to be sorted into two $n/2$-length arrays.
- Conquer: sort the two subproblems recursively using merge sort
- Combine: merge the two sorted subarrays to get the complete sorted array.

The "base case" is sorting an array of length 1, which needs no sorting. This stops the recursion.

Algorithm 3 Merge-Sort(A,p,r)

Input: An array A of nonnegative integers of length n, and two indices p, r.
Output: The sorted $A[p..r]$ subarray

```
if p < r then
    q ← ⌊(p + r)/2⌋
    MergeSort(A, p, q)
    MergeSort(A, q + 1, r)
    Merge(A, p, q, r)
```

MergeSort(A, p, r) sorts the elements in the subarray $A[p..r]$. If $p \geq r$, the subarray has at most one element, so it is already sorted. Otherwise, the divide step computes an index q that partitions $A[p..r]$ into two subarrays: $A[p..q]$, containing $\lceil (p+r)/2 \rceil$ elements, and $A[q+1, r]$, containing $\lfloor (p+r)/2 \rfloor$ elements[1], that are each sorted using recursive calls of the MergeSort procedure. The combine step is achieved with a call of the Merge(A, p, q, r) procedure, which we detail next. An initial call to MergeSort(A, 1, n) will sort the entire array.

The Merge procedure assumes that the subarrays $A[p..q]$ and $A[q+1..r]$ are sorted. It merges them to form a single sorted subarray that replaces the current subarray $A[p..r]$. Going back to the card-playing analogy, you can think of having two piles of sorted face-up cards on a table, such that the smallest card in each pile is on top. To sort the two piles into one, we compare the two showing cards, pick the smallest of them and place it face down on the table in the output pile. Once a card is removed from its pile, another one will show (unless it was the last card in the pile). We repeat the step until one input pile is empty. Then add the remaining cards in the other pile face down on the output pile. The following pseudocode implements this idea.

In order to not check at each step whether any of the piles is empty, we place a sentinel card at the bottom of each pile. We use ∞ as the sentinel value, so that when reaching the first sentinel card, it cannot be smaller than the card on the other pile. When the second sentinel card is unveiled, all the non-sentinel cards have already been placed onto the output pile. We stop once we have performed $r - p + 1$ basic steps, since we know this is the number of steps needed.

In detail, the Merge procedure works as follows. Lines 1 and 2 compute the respective lengths of the subarrays $A[p..q]$ and $A[q+1, r]$. Next we create arrays L and R ("left" and "right"), of lengths $n_1 + 1$ and $n_2 + 1$, respectively, in line 3 (the extra position in each array is for the sentinel). The two for loop of lines 4–7 copy the subarrays $A[p..q]$ and $A[q+1, r]$ into the arrays L and R, respectively. Lines 8–9 place the sentinels at the ends of L and R. Lines 10–18 of algorithm 4 perform the $r - p + 1$ basic steps to sort $A[p..r]$.

4.1 Correctness of MergeSort

Let us first prove that Merge is correct. The invariant of the for loop at lines 12-18 is that:

[1]The expression $\lceil x \rceil$ denotes the smallest integer greater than or equal to x, and $\lfloor x \rfloor$ denotes the greatest integer less than or equal to x

Algorithm 4 Merge(A,p,q,r)

```
1: n_1 ← q − p + 1
2: n_2 ← r − q
3: Create arrays L[0..n_1] and R[0..n_2]
4: for i ← 0 to n_1 − 1 do
5:     L[i] ← A[p + i]
6: for i ← 0 to n_2 − 1 do
7:     R[i] ← A[q + i + 1]
8: L[n_1] ← ∞
9: R[n_2] ← ∞
10: i ← 0
11: j ← 0
12: for k ← p .. r do
13:     if L[i] ≤ R[j] then
14:         A[k] ← L[i]
15:         i ← i + 1
16:     else
17:         A[k] ← R[j]
18:         j ← j + 1
```

> At the start of each iteration of the for loop of lines 12–18, the subarray $A[p..k-1]$ contains the $k-p$ smallest elements of $L[0..n_1]$ and $R[0..n_2]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A.

Initialization: Prior to the first iteration of the loop, we have $k = p$, so that the subarray $A[p..k-1]$ is empty. This empty subarray contains the $k - p = 0$ smallest elements of L and R, and since $i = j = 0$ and L and R are sorted, both L[i] and R[i] are the smallest elements of their arrays that have not been copied back into A.

Maintenance: Let us suppose that $L[i] \leq R[j]$. Then the smallest element not yet copied back into A is $L[i]$. We know that so far $A[p..k-1]$ contains the $k-p$ smallest elements. After line 14 copies $L[i]$ into $A[k]$, the subarray $A[p..k]$ will contain the $k-p+1$ smallest elements. Incrementing k (in the for loop update) and i (in line 15) reestablishes the loop invariant for the next iteration. If instead $R[j] < L[i]$, then lines 17–18 perform the appropriate action to maintain the loop invariant.

Termination: At termination, $k = r + 1$. By the loop invariant, the subarray $A[p..k-1]$ (which is $A[p..r]$), contains the $k - p = r - p + 1$ smallest elements of $L[0..n_1]$ and $R[0..n_2]$, in sorted order. The arrays L and R together contain $n_1 + n_2 + 2 = r - p + 3$ elements. All except the two largest (the sentinels) have been copied back into A.

Conclusion: Since at termination the subarray $A[p..r]$ is sorted, $\text{Merge}(A, p, q, r)$ correctly sorts the sorted subarrays $A[p..q], A[q+1, r]$.

Each of lines 1–3 and 8–11 takes constant time $\Theta(1)$. The for loops of lines 4–7 take $\Theta(n_1 + n_2) = \Theta(n)$ time, and there are n iterations of the for loop of lines 12–18, each of which takes constant time. Thus, Merge will run in $\Theta(n)$ time.

Now, let's focus on MergeSort. This time, we maintain a **recursion invariant** that any time MergeSort returns, it returns a sorted array:

At the start of each recursive call to MergeSort(A, p, r), the subarray $A[p..r]$ contains the same elements as in the original input (unsorted). After the procedure completes, $A[p..r]$ is sorted.

Initialization (Base Case - trivial sort): The base case occurs when MergeSort(A, p, r) is called with $p \geq r$, meaning that the subarray contains only one element or no elements. A single-element array is already sorted by definition. Since no further operations are needed, the loop invariant trivially holds.

Maintenance (Recursive setp): Recursive Calls Preserve the Invariant

Now, we assume that before calling MergeSort(A, p, r), the loop invariant holds for smaller subarrays.

Step 1: Recursive sorting of halves. The algorithm recursively calls itself:

$q \leftarrow \lfloor (p+r)/2 \rfloor$

MergeSort$(A, p, q) \rightarrow$ sorts left half

MergeSort$(A, q+1, r) \rightarrow$ sorts right half

By the inductive hypothesis, these recursive calls correctly sort the left and right halves before merging them.

Step 2: merging the two subarrays. We proved that the Merge procedure is correct. Since no elements are lost or duplicated, the original array's elements are preserved and sorted at the end of MergeSort.

Termination (Conclusion): MergeSort terminates when the recursive calls reach the base case, at which point each small subarray is sorted. Merge operations combine all subarrays into a fully sorted array. Eventually, the final merge step produces a sorted version of the original array. Thus, when MergeSort(A, 1, n) completes, the entire array A is sorted.

Conclusion: Since the recursion invariant holds at termination, MergeSort correctly sorts any input array.

4.2 Analyzing Divide-and-conquer algorithms

When an algorithm contains a recursive call to itself, we can often describe its running time by a **recurrence equation** which describes the overall running time on a problem of size n in terms of the running time on smaller inputs. We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.

Let $T(n)$ be the running time on a problem of size n. For a small enough problem size (say $n \leq c$ for some constant c), the straightforward solution takes constant time, which we write as $\Theta(1)$. Suppose that the divide step of the problem yields a subproblems, each of which is $1/b$ of the size of the original. (For merge sort, $a = b = 2$, but there are many divide-and-conquer algorithms in which $a \neq b$, e.g. Karatsuba's algorithm for fast multiplication.) It takes time $T(n/b)$to solve one subproblem of size n/b, so it will take $aT(n/b)$ to solve a of them. If we take $D(n)$ time to divide the problem into subproblems and $C(n)$ time to combine the solutions of the subproblems into the solution of the original problem, we get the recurrence:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n), & \text{otherwise} \end{cases}$$

Analysis of MergeSort

To simplify our analysis, we will assume that n is a power of two. Then each divide step generates two subproblems of size exactly $n/2$. When we have n $>$ 1 elements, we break down the running time as follows:

Divide: The divide step computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.

Conquer: We recursively solve two subproblems, each of size $n/2$, contributing together $2T(n/2)$ to the running time.

Combine: We have seen the Merge procedure on an n-element subarray takes time $\Theta(n)$, and so $C(n) = \Theta(n)$.

Adding the divide and combine steps, we get $\Theta(1) + \Theta(n) = \Theta(n)$. Thus, we obtain the recursion:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1 \\ 2T(n/2) + \Theta(n), & \text{if } n > 1 \end{cases}$$

You can see in [1], Chapter 4, a proof that $T(n)$ is $n \lg n$ ($\lg n$ stands for $\log_2 n$). For the scope of this course it is enough to intuitively understand why this is.

Let c denote the constant time required to solve problems of size 1, as well as the time required for the divide and combine steps per array element. We will rewrite the recursion as:

$$T(n) = \begin{cases} c, & \text{if } n \leq 1 \\ 2T(n/2) + cn, & \text{if } n > 1 \end{cases}$$

Figure 1 shows how we can solve the recurrence. Part (a) shows the initial $T(n)$, which adds the cn marked in the root and expands to two $T(n/2)$ subtrees in (b), each of which further expands to two $T(n/4)$ trees, having the $c \cdot size$ term in the intermediary nodes. The expansion continues until we only have problems of size 1, which have the cost c. Part (d) shows the resulting **recursion tree**.

Now we have to add the costs on each level of the tree. The root has cost cn, the level below it $c(n/2) + c(n/2) = cn$, the next level $4c(n/4) = cn$, and so on until the leaves level. In general, level i from the root has 2^i nodes, each contributing $c(n/2^i)$ to the sum, for a total of $2^i c(n/2^i) = cn$ per level. The recursion tree has $\lg n+1$ levels, where n is the number of leaves, corresponding to the input size ($n = 2^m$, each level except the root and leaves represents a division by two, up to 2^0).

So the recursion tree has $\lg n + 1$ levels, each costing cn, for a total cost of $cn(\lg n + 1) = cn\lg n + cn$. Ignoring the low-order term and the constant c, we get $\Theta(n\lg n)$.

Number Multiplication

Suppose we are given two large numbers, each of n digits, and we want to multiply them. We could use the multiplication algorithm we learned in school. To multiply 1234 and 5678, we multiply 8 by each digit of the first number, then 7 by each digit of the first number and so on, adding the partial results to get the final answer. We have $n \times n$ multiplications, so a quadratic algorithm.

Could we multiply the numbers differently?

We can write x =1234 as 12×100+34, and y =5678 as $56 \times 100 + 78$. Multiplying x and y becomes $12 \times 56 \times 10^4 + 12 \times 78 \times 10^2 + 34 \times 56 \times 10^2 + 34 \times 78$.

More generally, if we write $x = a \times 10^{n/2} + b$ and $y = c \times 10^{n/2} + d$, we get

$$x \times y = (a \times 10^{n/2} + b) \times (c \times 10^{n/2} + d) = ac \times 10^n + (ad + bc) \times 10^{n/2} + bd.$$

Now we can split this problem into four subproblems, where each subproblem is similar to the original problem, but with half the digits. The recursion is $T(n) = 4T(n/2) + O(n)$ because we are splitting the problem into four subproblems with half the digits, plus some additions with linear cost.

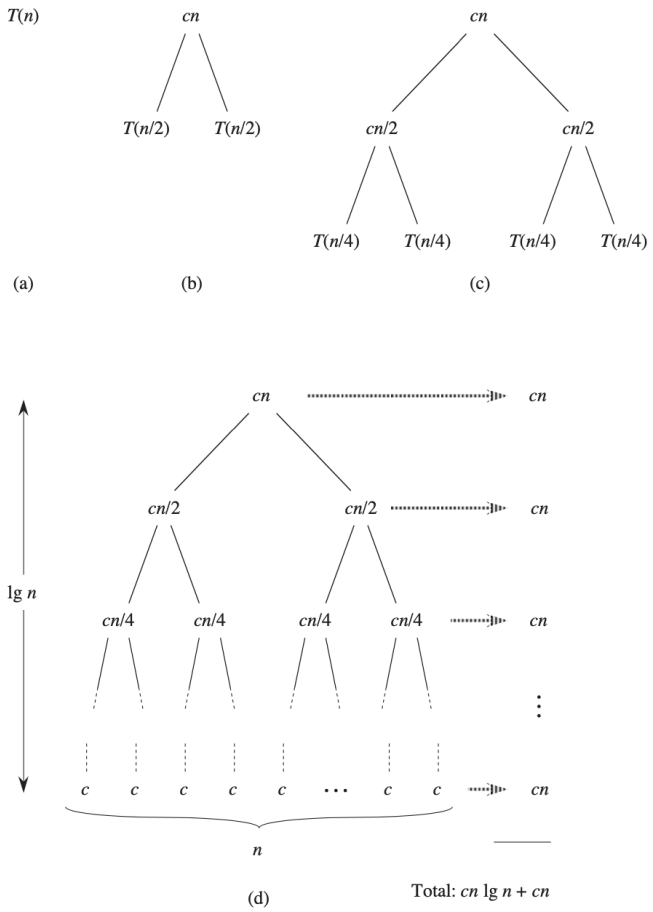

Figure 1: Figure 2.5 of CLRS. Buildinng a recursion tree for the recurrence $T(n) = 2T(n/2) + cn$

We make lg$n$ splits, arriving at $T(n) = 2^{2\lg n}T(\frac{n}{2^{\lg n}}) = n^2$, so we haven't really improved on the execution time. However, **Karatsuba** (see [2]) found a better algorithm by noticing that we need the sum $ad+bc$, not the actual values of ad and bc. Note that $(a+b)\times(c+d) = ac+ad+bc+bd$, and we already need ac and bd. So, to compute $ad+bc$, Karatsuba did $(a+b)\times(c+d) - ac - bd$, thus needing only 3 $n/2$-digit multiplications (3 subproblems instead of 4).

Counting the number of multiplications of pairs of 1-digit numbers required to compute the product of two n-digit numbers, Karatsuba's algorithm gives

$T(n) = 3T(n/2) = 3^2T(n/4) = ... = 3^{\lg n}T(\frac{n}{2^{\lg n}}) = 2^{(log_2 3)(log_2 n)} = n^{log_2 3} \leq n^{1.6}$.

Further improvements have been made to the multiplication problem.

- **Toom–Cook (1963)**: $O(n^{\log_3 5}) \approx O(n^{1.465})$
- **Schönhage–Strassen (1971)**: $O(n \log n \log \log n)$
- **Fürer (2007)**: $O(n \log n\, 2^{O(\log^* n)})$
- **Harvey–van der Hoeven (2019)**: $O(n \log n)$

5 Asymptotic Notation

We will use asymptotic notation primarily to describe the running times of algorithms. However, asymptotic notation actually applies to functions. Recall that we characterised the running time of InsertionSort as $an^2 + bn + c$, for some constants a, b, c. We can abstract some details (constants, lower order terms) of the quadratic function and express the running tine as $\Theta(n^2)$.

Figure 2 graphically shows examples of the asymptotic notations O, Ω, and Θ .

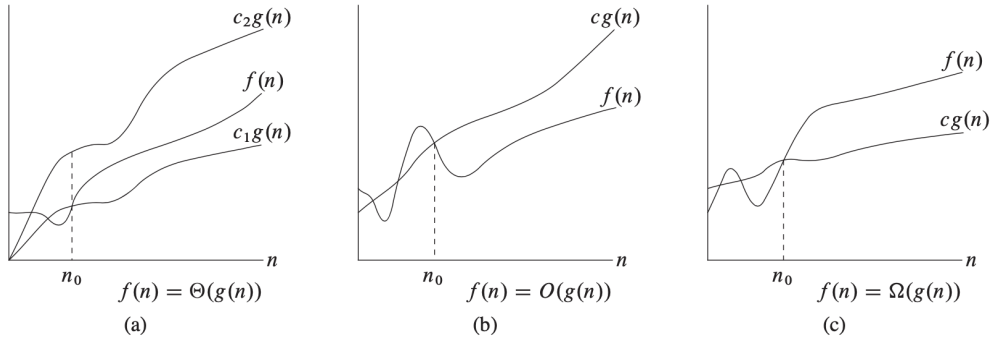

Figure 2: Figure 3.1 of CLRS. Graphic examples of the O, Ω, Θ notations.

"Big-Oh(O)" Notation:

The Big-Oh notation gives an **asymptotic upper bound**. $O(g(n)) = \{f(n) | \exists n_0, c > 0 \text{ s.t. } 0 \leq f(n) \leq cg(n), \text{ for all } n \geq n_0\}$.

For running time $T(n)$ we have that

$$T(n) = O(f(n)) \Leftrightarrow \exists n_0, c > 0 \text{ s.t. } 0 \leq T(n) \leq cf(n), \forall n \geq n_0.$$

"Big Omega(Ω)" Notation

Ω-notation provides an **asymptotic lower bound**. $\Omega(g(n)) = \{f(n) | \exists n_0, c > 0 \text{ s.t.} 0 \leq cg(n) \leq f(n), \forall n \geq n_0\}$.

We say $T(n)$ is $\Omega(f(n))$ if, when n gets large, $T(n)$ grows at least as slowly as $f(n)$.

$$T(n) = \Omega(f(n)) \Leftrightarrow \exists n_0, c > 0 \text{ s.t. } 0 \leq cf(n) \leq T(n), \forall n \geq n_0.$$

"Big Theta(Θ)" Notation

The Θ-notations gives an **asymptotically tight bound** for a function $f(n)$, meaning a function $g(n)$ that is both an upper and a lower bound for $f(n)$ with some constant factors, and for values $n \geq n_0$.

$\Theta(g(n)) = \{f(n) | \exists n_0, c_1, c_2 > 0 \text{ s.t. } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0\}$.

Running time $T(n)$ is $\Omega(f(n))$ if and only if (shorthand, iff) $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$:

$$T(n) = \Theta(f(n)) \Leftrightarrow \exists n_0, c_1, c_2 > 0 \text{ s.t. } 0 \leq c_1 f(n) \leq T(n) \leq c_2 f(n), \forall n \geq n_0.$$

6 Turing (Cook) problem reductions

We say that a problem P polynomially reduces to a solvable problem Q, and write P $\propto$ Q, if we can design an algorithm for P using an algorithm for Q, as follows:

1. let p be an instance of P;
2. preprocess in polynomial time the input p to obtain an instance q (or several instances) of Q;
3. call an algorithm for Q, possibly several times (but a polynomial number of times);
4. postprocess the output(s) given by Q in polynomial time to get the answer P(p).

If the (pre+post)processing time requires O(g(n)) time, then we write P $\propto_{g(n)}$ Q. Reducing problem P to problem Q means that P is no harder than Q. We can use problem reductions to prove algorithmic upper bounds.

There exist other types of reductions, like Karp reductions, but more on that in the lectures on NP-completeness. The main differences are Karp reductions are made on decision problems, a single call is permitted to the algorithm that solves Q, and the output cannot be post-processed.

Example: SET-INCLUSION$\propto$SORTING

SET-INCLUSION
INSTANCE: Two finite sets of natural numbers $A = \{a_0, a_1, ..., a_{n-1}\}$ and $B = \{b_0, b_1, ..., b_{m-1}\}$, with $n \leq m$
QUESTION: $A \subseteq B$?

Reduction algorithm

1. Compute A'=Sort(A) and B'=sort(B)
2. Go through A' and B', left to right
3. If $A'_i = B'_j$, advance in both arrays
4. If $A'_i > B'_j$, advance in B' to search for the next element of A'
5. If $A'_i < B'_j$, then $A'[i]$ does not appear in B, A is not a subset of B
6. If all elements of A' are found in B', A is a subset of B.

References

[1] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.

[2] A. A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics Doklady*, 7:595–596, 1963.

Algorithm 5 SetInclusion(A, B)

```
A' ← Sort(A)
B' ← Sort(B)
i ← 0
j ← 0
while i < n and j < m do
    if A'_i = B'_j then
        i ← i + 1
        j ← j + 1
    else if A'_i > B'_j then
        j ← j + 1
    else
        return NO
if i = n then
    return YES
else
    return NO
```